Bradley Capstone 2021
Code Documentation

Abilities (Disasters)

Relevant Classes/Files/Directories

- Scripts/Abilities/IAbility.cs
- Scripts/Abilities/ITriggerAbility.cs
- Scripts/Abilities/IColliderAbility.cs
- Scripts/Abilities/Placement/IAbilityPlacement.cs
- Scripts/Abilities/Placement/StandardPlacement.cs
- Scripts/Abilities/Placement/VectorPlacement.cs
- Scripts/Abilities/AbilityData.cs
- ScriptableObjects/Ability Objects/Ability Data
- ScriptableObjects/Ability Objects/Cooldown Channels

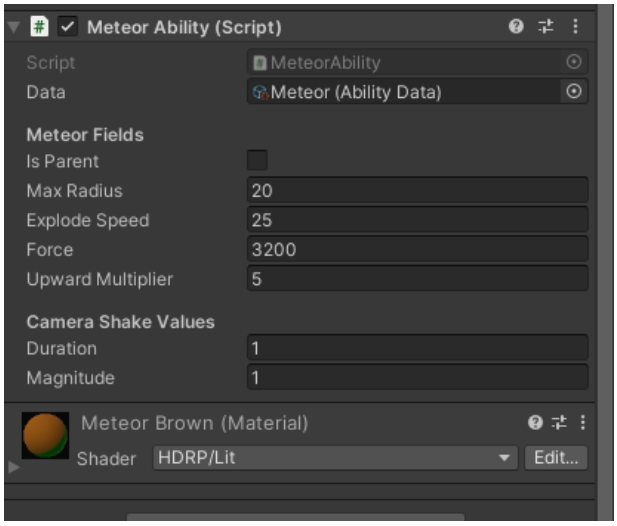
Concept Overview

An ability, also known as a disaster, is a special power that the player can use to push, pull, or otherwise move their golf ball around the course.

Please note that ability and disaster will be used synonymously throughout this document.

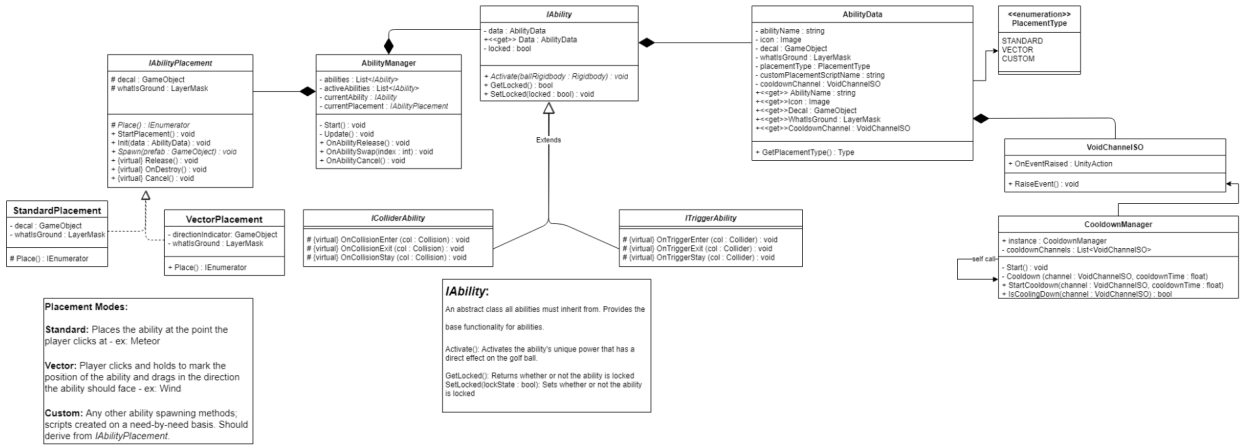
Creating a new Ability Class

- Navigate to Scripts/Abilities/Concrete Abilities, right click, and create a new C# Class.
- Each ability must derive from either ITriggerAbility if the ability uses triggers to interact with the golf ball, or from IColliderAbility if the ability uses colliders to interact with the golf ball.
- From there, you'll fill in the abstract and virtual methods (if you choose to make use of them) to define the ability's behavior.



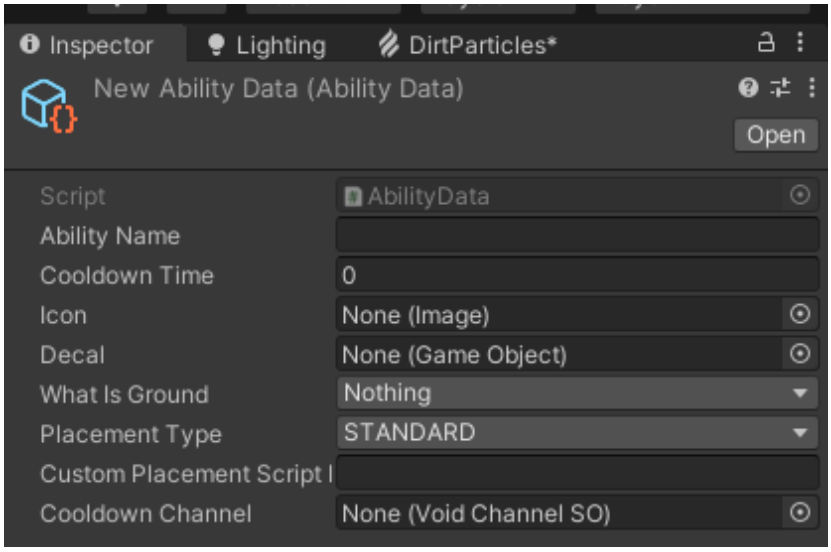
(A complete ability)

Ability Class Object Diagram (Subject to Change)



Ability Data

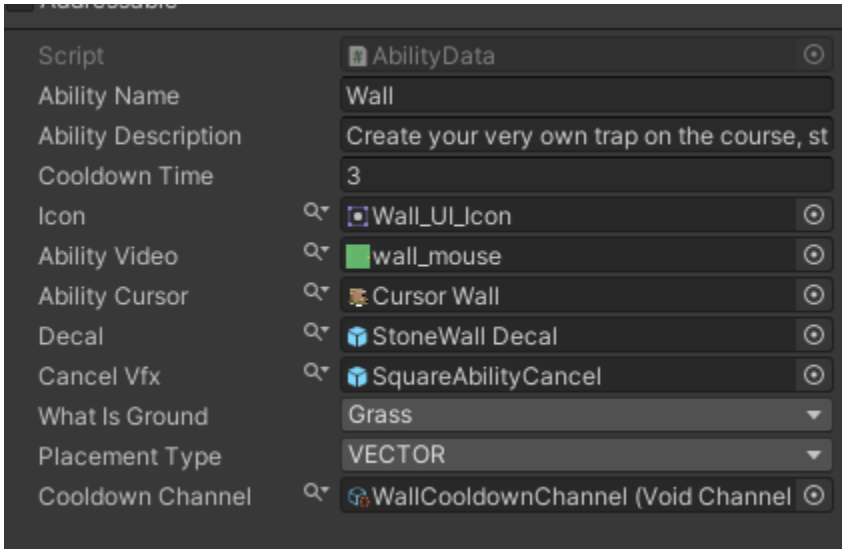
Each ability must have an AbilityData asset associated with it. AbilityData is a ScriptableObject that acts as a data container to encapsulate an ability's metadata. To create a new AbilityData ScriptableObject asset, navigate to ScriptableObjects/Ability Objects/, right click in an empty space, hover over "Create/Abilities", then left click on "Ability Data". An AbilityData asset will be created that you can modify per the ability's requirements.



(A newly created AbilityData asset)

AbilityData contains the following modifiable fields:

- Ability Name : a string denoting the ability's name.
- Ability Description : a string describing the ability's function.
- Cooldown Time : the time in seconds until the ability can be used again after using it.
- Icon : An Image of the ability's icon to be displayed on the UI.
- Ability Video: the video clip that showcases the ability in use. Acts as a tutorial.
- Ability Cursor: the cursor to use when this ability is active.
- Decal : the GameObject that will be spawned underneath the cursor to visualise the location and direction/radius the ability will spawn with.
- Cancel Vfx: the GameObject that is spawned when the ability is cancelled.
- What Is Ground : a LayerMask denoting which layers are acceptable to place the ability on.
- Placement Type : an enum denoting how the ability will be placed in the game world (see below).
- Custom Placement Script Name : The name of the custom placement script that will be added as a component in place of other, predefined placement scripts. Only applicable if Placement Type is CUSTOM.
- Cooldown Channel: Each Ability has a unique cooldown channel to subscribe to.

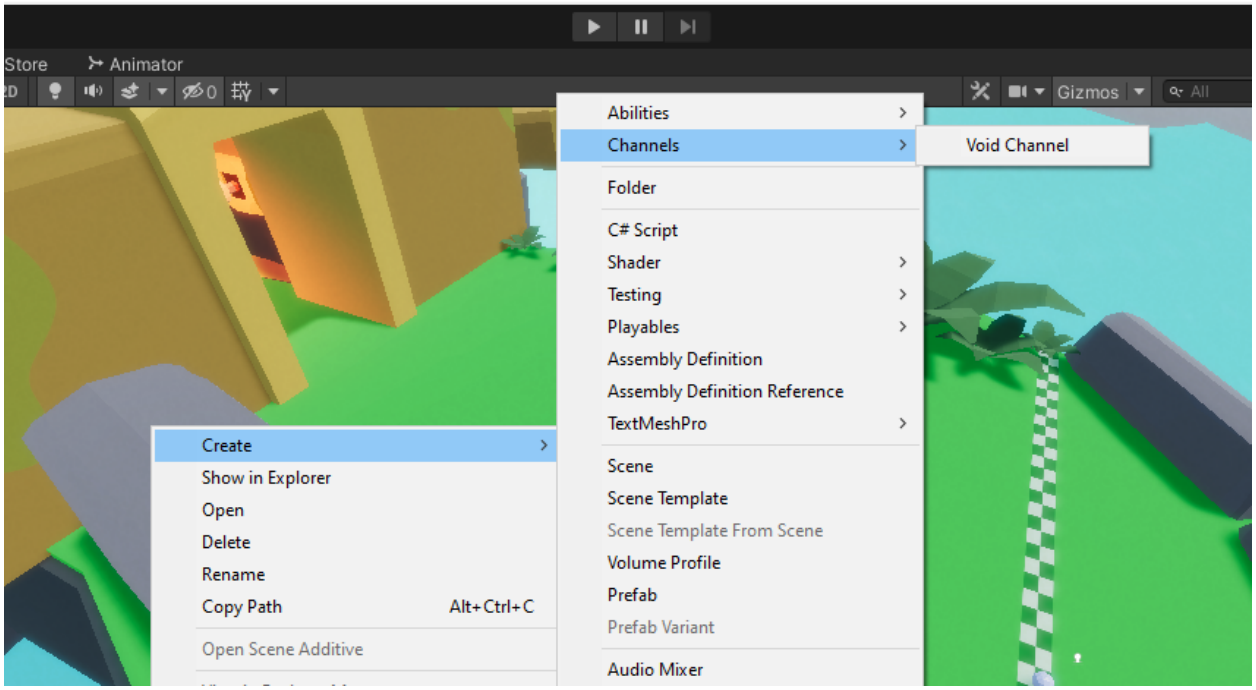


(A complete AbilityData asset)

Ability Cooldown Channels

Each ability has a unique cooldown channel to subscribe to. Ultimately, it is a Void Channel that is able to receive event subscribers and have its event invoked via the RaiseEvent() method. The event raised resets the ability to be able to be placed again.

In order to create a new cooldown channel, navigate to the ScriptableObjects/Ability Objects/Cooldown Channels directory, right click in an empty space, hover over "Create/Channels/Void Channel" and left click. A new Void Channel ScriptableObject asset will be created. This asset should be renamed to "*ability_name*CooldownChannel" and be dragged into the Cooldown Channel field in the ability's AbilityData asset.



Ability Placement

Each ability has an IAbilityPlacement field that determines how the player will place it. There are currently 4 types of ability placements, shown below.

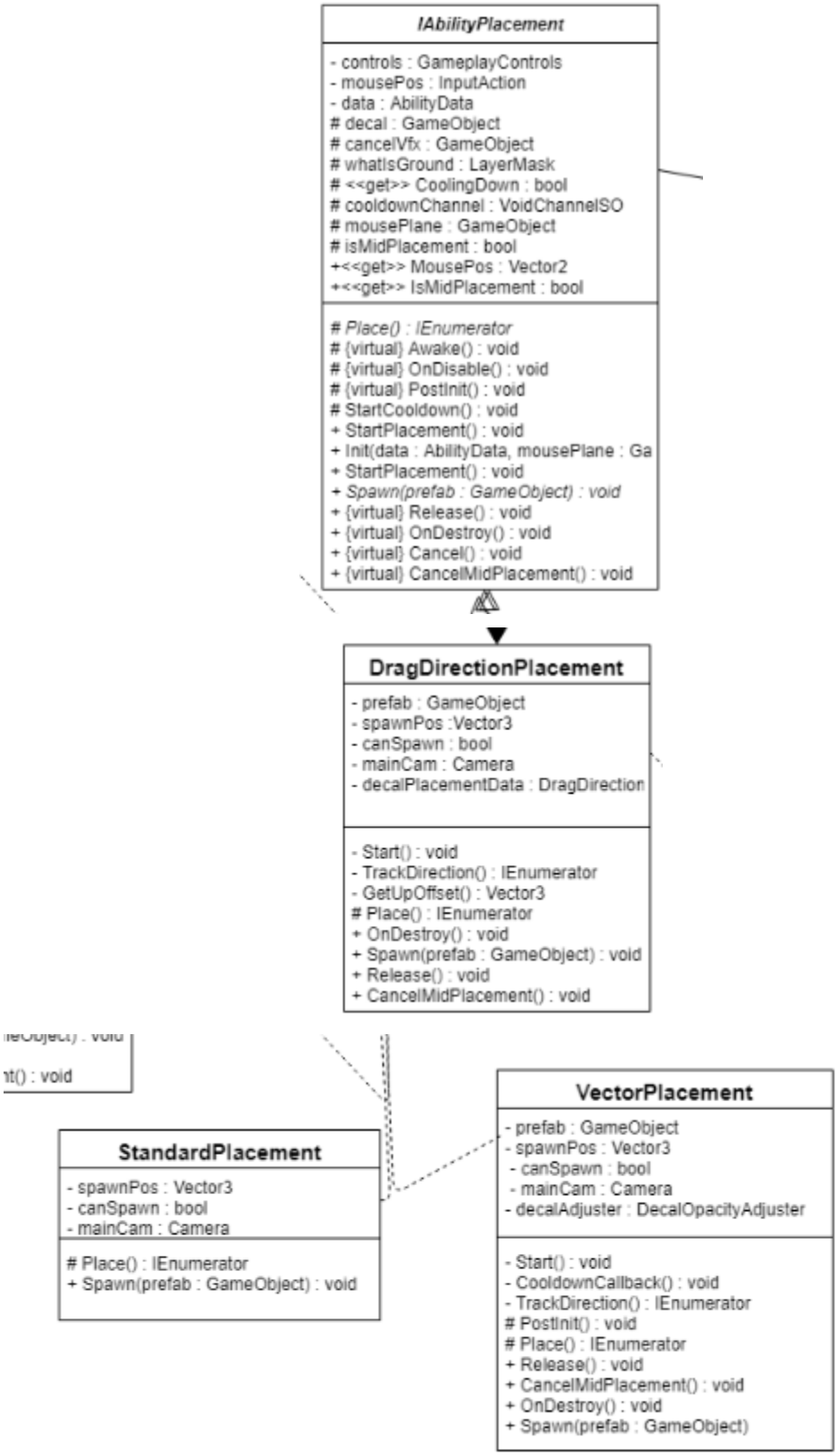
Placement Modes:

Standard: Places the ability at the point the player clicks at. example: Meteor

Vector: Player clicks and holds to mark the position of the ability and drags in the direction the ability should face.

DragDirection: Player clicks and holds to set an anchor point then drags in the direction the ability should face in when spawned. example: Wind

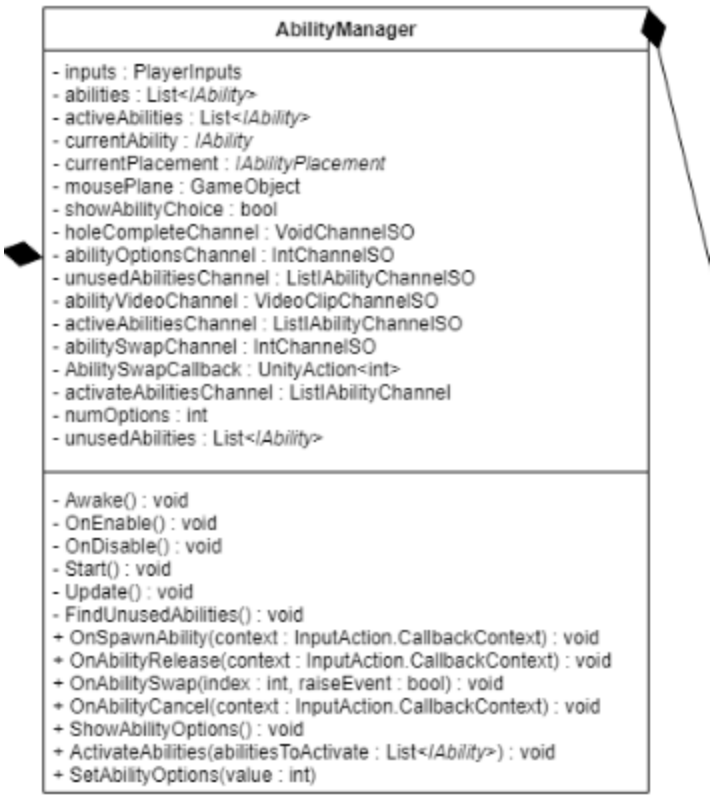
Custom: Any other ability spawning methods; scripts created on a need-by-need basis. Should derive from *IAbilityPlacement*.


IAbilityPlacement
- controls : GameplayControls
- mousePos : InputAction
- data : AbilityData
decal : GameObject
cancelVfx : GameObject
whatIsGround : LayerMask
<<get>> CoolingDown : bool
cooldownChannel : VoidChannelSO
mousePlane : GameObject
isMidPlacement : bool
+<<get>> MousePos : Vector2
+<<get>> IsMidPlacement : bool
Place() : IEnumerator
{virtual} Awake() : void
{virtual} OnDisable() : void
{virtual} PostInit() : void
StartCooldown() : void
+ StartPlacement() : void
+ Init(data : AbilityData, mousePlane : Ga
+ StartPlacement() : void
+ Spawn(prefab : GameObject) : void
+ {virtual} Release() : void
+ {virtual} OnDestroy() : void
+ {virtual} Cancel() : void
+ {virtual} CancelMidPlacement() : void
DragDirectionPlacement
- prefab : GameObject
- spawnPos :Vector3
- canSpawn : bool
- mainCam : Camera
- decalPlacementData : DragDirection
- Start() : void
- TrackDirection() : IEnumerator
- GetUpOffset() : Vector3
Place() : IEnumerator
+ OnDestroy() : void
+ Spawn(prefab : GameObject) : void
+ Release() : void
+ CancelMidPlacement() : void
StandardPlacement
- spawnPos : Vector3
- canSpawn : bool
- mainCam : Camera
Place() : IEnumerator
+ Spawn(prefab : GameObject) : void
VectorPlacement
- prefab : GameObject
- spawnPos : Vector3
- canSpawn : bool
- mainCam : Camera
- decalAdjuster : DecalOpacityAdjuster
- Start() : void
- CooldownCallback() : void
- TrackDirection() : IEnumerator
PostInit() : void
Place() : IEnumerator
+ Release() : void
+ CancelMidPlacement() : void
+ OnDestroy() : void
+ Spawn(prefab : GameObject)

- Standard Placement : A simple “click and spawn” placement type; ability will spawn on top of the player’s cursor. The decal follows the player’s cursor.
- VectorPlacement : Used when a direction to spawn the ability is required. Player clicks and holds the designated ability spawn button, then rotates their mouse to alter the ability’s direction. The decal is instantiated over the player’s cursor and is rotated with the mouse to visualize the location and direction the ability will spawn with.
- DragDirectionPlacement: Used to spawn in a disaster in a direction. Similar to VectorPlacement, but more information is provided to the player. A decal is anchored to the initial click point, and a secondary decal follows the cursor. A line connects the anchor point and the decal underneath the cursor.
- Custom : Custom placement types should be created in Scripts/Abilities/Concrete Abilities directory. Once there, create a new C# script and inherit from IAbilityPlacement. If using a custom placement type, **do not forget** to select the CUSTOM placement type on the AbilityData asset and provide the class’s name in the “Custom Placement Script Name” field.
    - Place() : a coroutine called when the ability is initially swapped to.
    - Spawn() : a method invoked when the player presses the designated ability spawn button.
    - Release() : a method invoked when the player releases the designated ability spawn button.
    - Cancel(): a method invoked when the player cancels the ability’s placement.

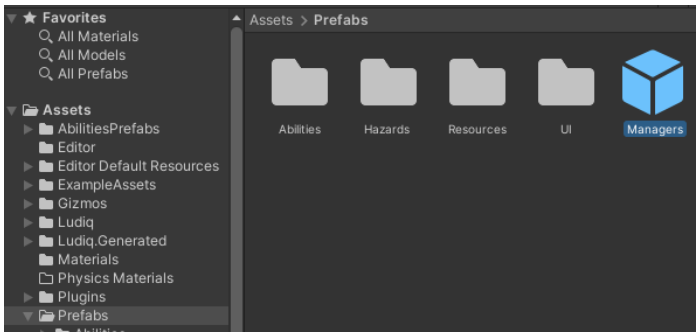
Ability Manager

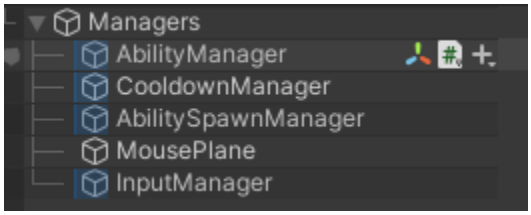
The AbilityManager is a manager component that must be present in the scene. It holds two lists that contain all of the abilities and all of the abilities the player has equipped respectively. In addition, it holds the currently equipped ability.



Setting up Disasters in a new Level

Dragging the Managers prefab into your scene should allow for the basics of disaster spawning. The Managers prefab includes the AbilityManager, CooldownManager, AbilitySpawnManager, and MousePlane used to spawn disasters, and it also includes the InputManager, which has the necessary components for setting up inputs in your scene. If you are still having problems with this, message TJ or Kyle for help!

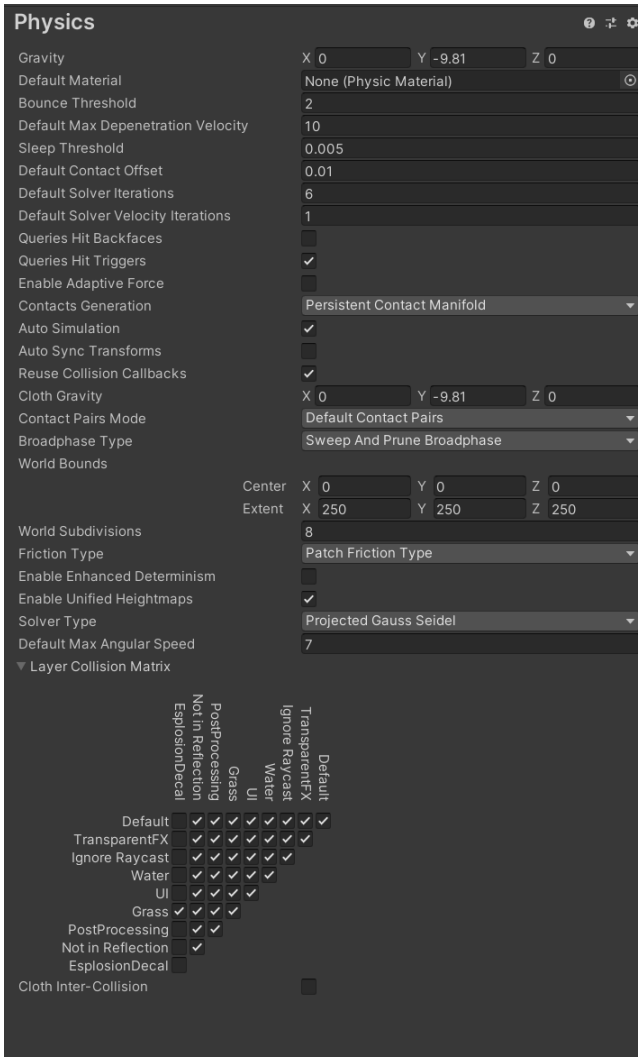




Physics Funtimes

Document for common physics tools, references, and resources:

Default Physics Values in Build Settings > Physics. Try to only use this for individual experimenting and not implementation.



Resources courtesy of Matt:
https://learn.unity.com/tutorial/physics-best-practices
https://digitalopus.ca/site/using-rigid-bodies-in-unity-everything-that-is-not-in-the-manual/
https://docs.unity3d.com/ScriptReference/ForceMode.html
https://www.youtube.com/watch?v=pTz3LMQpvfA
https://learn.unity.com/tutorial/physics-best-practices#
https://digitalopus.ca/site/using-rigid-bodies-in-unity-everything-that-is-not-in-the-manual/
https://www.youtube.com/watch?v=pTz3LMQpvfA&ab_channel=Unity

Regarding the meteor:
When direct-impacting the ball, it launches straight upwards. When hitting the ball a little bit away, it has a stronger launch force the closer it is to the ball(i.e., a gradient). May want to look into an arrow or something to show the launch gradient?

Ability Selection

Relevant Classes/Files/Directories

- Scripts/Scripts/UI/AbilityChoices/PlayerChosen/AbilityChoiceController.cs
- Scripts/UI/AbilityChoices/PlayerChosen/AbilityChoice.cs
- Scripts/Scripts/UI/AbilityChoices/Random/AbilityChoiceController.cs
- Scripts/UI/AbilityChoices/Random/AbilityChoice.cs
- Scripts/Abilities/Managers/AbilityManager
- Prefabs/UI/AbilityChoiceSelection
- Prefabs/UI/AbilityChoiceRandom

Concept Overview

Player Chosen:

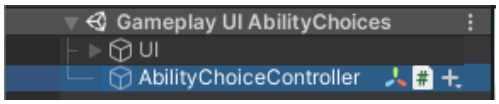
After each hole, the player will have the option to select new abilities to use from a pool of 3 abilities. The pool will be 3 abilities randomly taken from the unused abilities. After selecting their new abilities (up to 2 for now), the player will confirm their selection and gain access to those abilities at the start of the next hole. If there are not enough unused abilities to fill out the number of options selected in AbilityChoiceController, then options will be taken from used abilities as well.

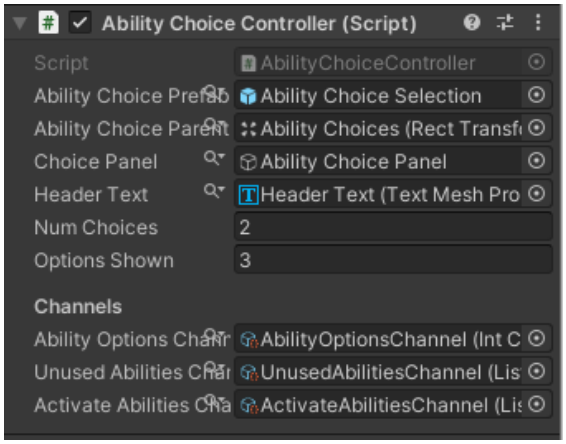
Random:

After each hole, the player will be given random new abilities taken from the unused abilities. The player will click the Confirm button to gain these abilities and move on to the next hole. If there are not enough unused abilities to fill out the number of new abilities in AbilityChoiceController, then options will be taken from used abilities as well.

AbilityChoiceController (Player Chosen)

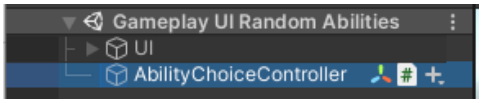
- The number of abilities the player can choose to move forward with and the number of options they are shown can be set in the AbilityChoiceController, located on the AbilityChoiceController object in the Gameplay UI AbilityChoices scene

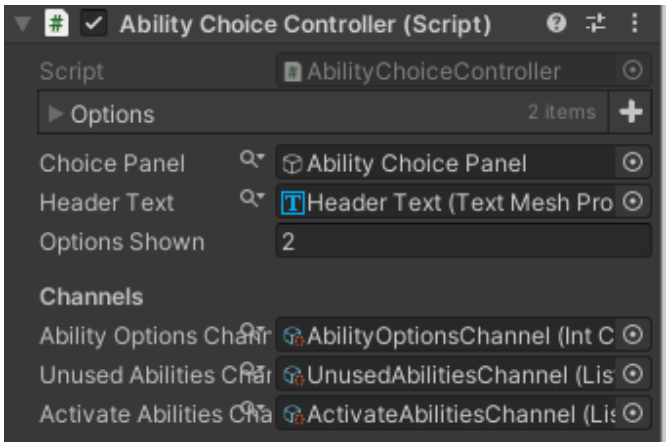




AbilityChoiceController (Random)

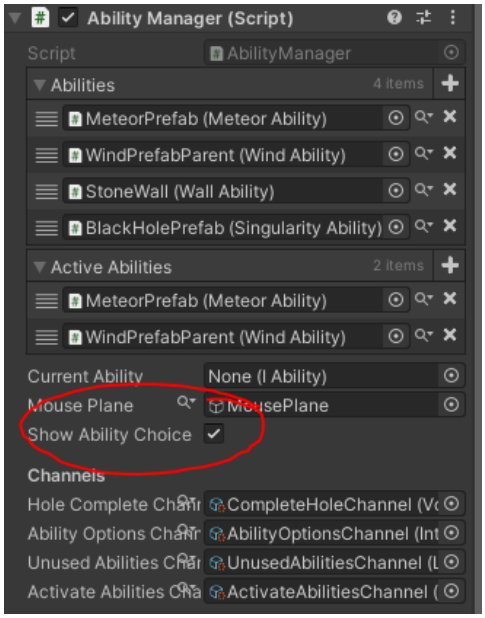
- The number of new abilities the player gets can be set using the Options Shown variable in the AbilityChoiceController, located on the AbilityChoiceController object in the Gameplay UI Random Abilities scene





Turning off the Ability Choice popup

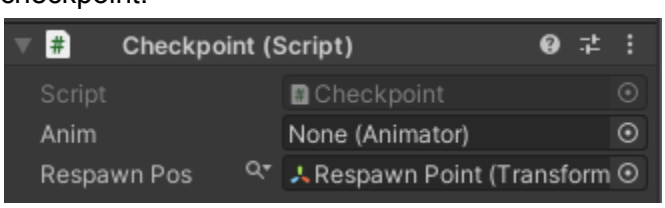
For both types of ability choice, you can turn off the popup at the beginning of the hole using the Show Ability Choice variable on the AbilityManager. Setting this to true will show the ability choice popup at the start of the level, and setting it to false will not.

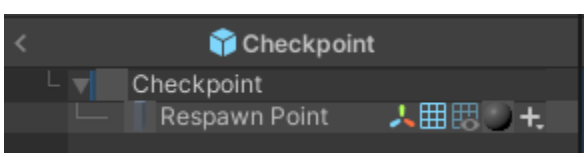


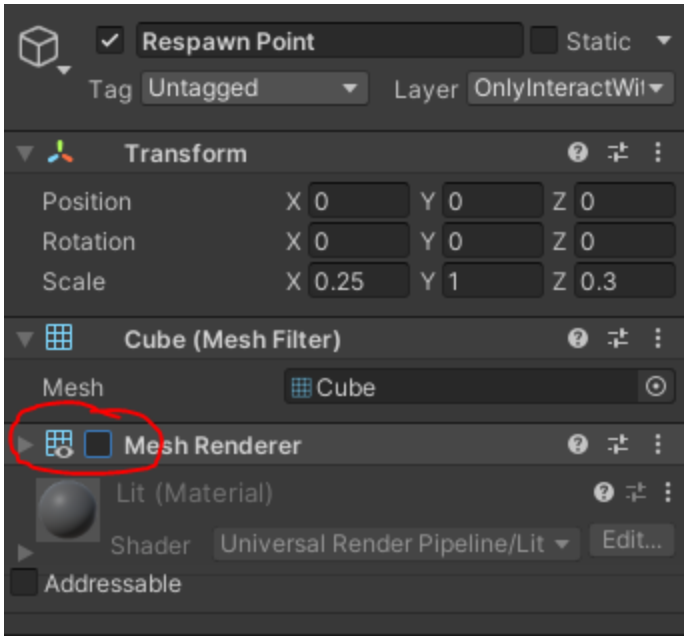
Checkpoints

The checkpoint prefab is located in the Prefab folder. Checkpoints can be placed by dragging one into the scene and positioning it where you want the checkpoint to be in the level. The mesh renderer component of the checkpoint can be turned on to assist in making sure placement is correct.

The exact point the ball respawns at is controlled by the position of the Respawn Point object, which is a child of the Checkpoint prefab. Like the checkpoint object, the mesh renderer of the respawn point can be turned on to help visualize where the ball will respawn once it hits this checkpoint.






Respawn Point
Static
Tag Untagged
Layer OnlyInteractWit
Transform
Position X 0 Y 0 Z 0
Rotation X 0 Y 0 Z 0
Scale X 0.25 Y 1 Z 0.3
Cube (Mesh Filter)
Mesh Cube
Mesh Renderer
Lit (Material)
Shader Universal Render Pipeline/Lit
Edit...
Addressable

The DragDirectionDecalData Component

Relevant Classes/Files/Directories

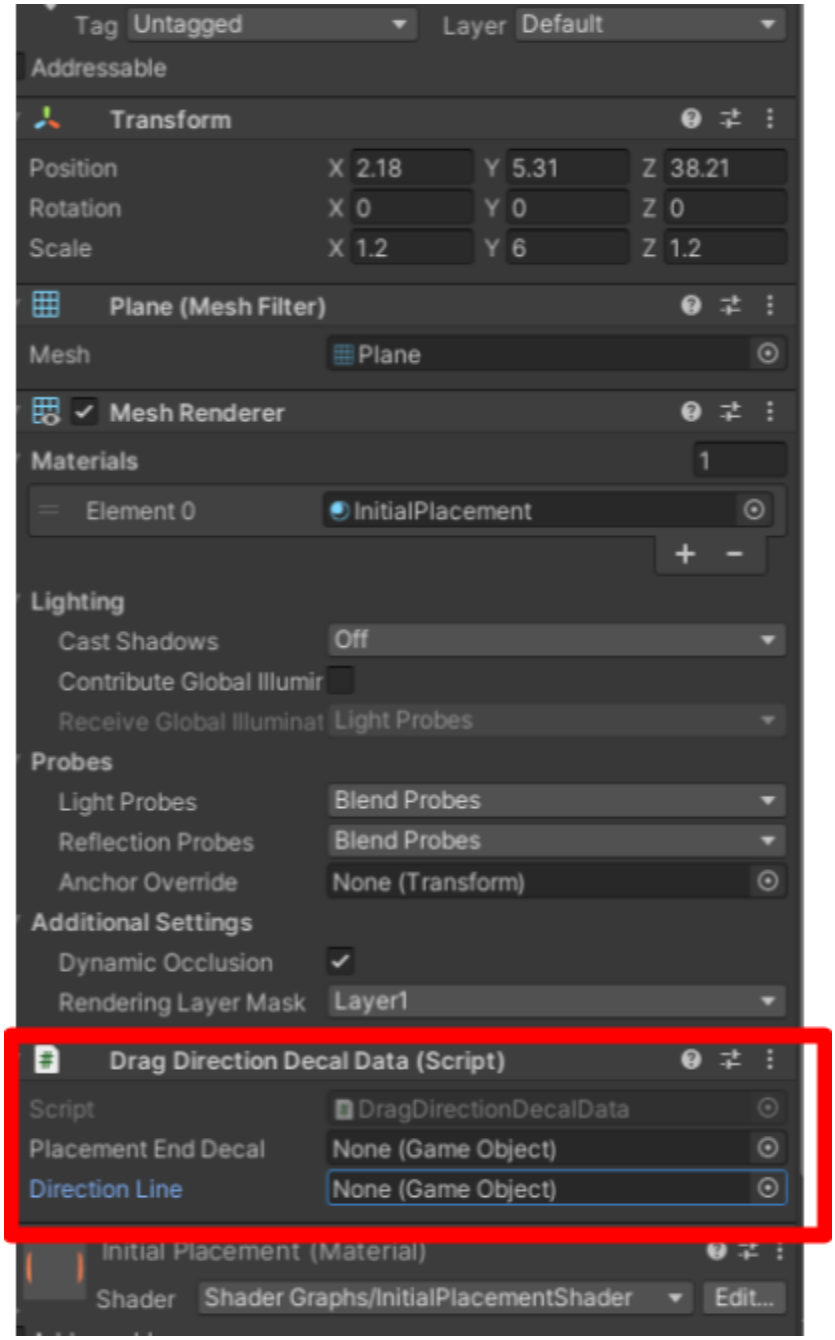
- Prefabs/Abilities/Decals
- Scripts/Abilities/DragDirectionDecalData.cs
- Scripts/Abilities/Placement/DragDirectionPlacement.cs
- Prefabs/VFX Components/VFX C Disasters/PlacementClarity/Raw Assets/DottedLineInformationGatherer.cs

Concept Overview

The DragDirectionDecalData component is a data container with quality of life functionality that enables multiple decals to be instantiated and moved around in the game world. Compared to the Vector Placement type, the purpose of the Drag Direction placement type is to give the player more information about the direction a disaster will be spawned in; the DragDirectionDecalData component aids in this process.

Adding the Component

Every decal for a disaster being placed with the Drag Direction placement type requires the DragDirectionDecalData component. Navigate to the Decals folder (see above for full directory) and left click on the decal you wish to use for the Drag Direction placement type. Navigate to the Inspector, left click on “Add Component” at the bottom of the Inspector, and add the DragDirectionDecalData component.



An newly-added DragDirectionDecalData component

Customizing the Decal

Before customizing the decal to work with Drag Direction placement, it is important to understand how Drag Direction placement works.

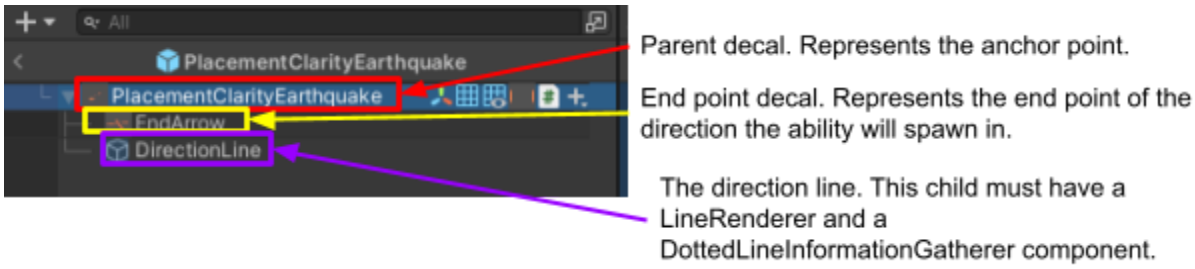
The Drag Direction placement type works as follows: the player left clicks and holds to begin the placement process. This initial click point will be referred to as the **anchor point**. Next, the player drags their mouse in the direction they wish to spawn the disaster in. A new decal follows the cursor around, and a line connects this new decal to the anchor point. This new decal will be referred to as the **end point** or the **placement end decal.**

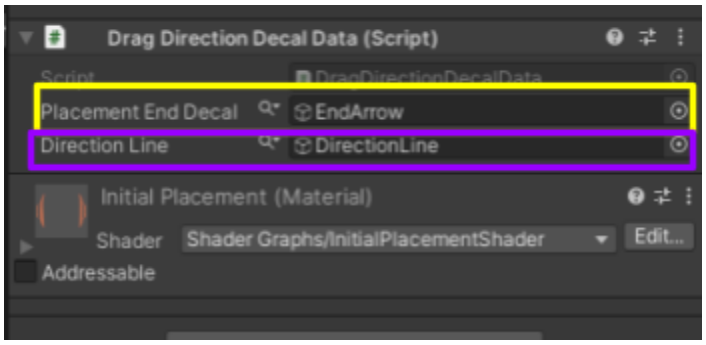
There are two fields that must be assigned for the component to work properly. The first is the Placement End Decal. This is a GameObject that represents the end point of the placement process. The second field is the Direction Line. This is a GameObject that represents the line that connects the anchor point to the end point.

The decal GameObject should have two children, one being the placement end decal and the other being the direction line. The parent GameObject should represent an image that will visualize the anchor point, the placement end decal GameObject should represent an image that will visualize the end point, and the direction line should have a LineRenderer component *and* a DottedLineInformationGatherer component that will draw the line between the anchor point and the end point. For more information on setting this child up, see the section titled “The Dotted Line Information Gatherer Component” below.

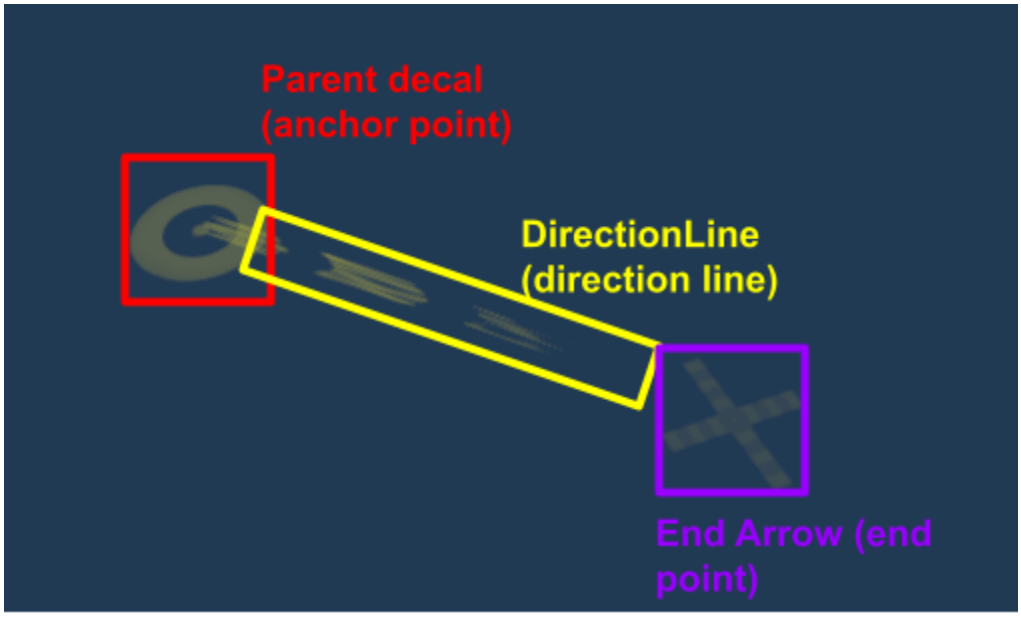
Once the parent decal has both children set up, drag in the child representing the end point into the Drag Direction Decal Data’s “Placement End Decal” field and the child representing the direction line into the “Direction Line” field.

Now that the decal has been customized, don’t forget to add it to the disaster’s AbilityData asset!





A completed DragDirectionDecalData component.



Example of a completed Drag Direction Placement decal.

The Dotted Line Information Gatherer Component

This component must be added to the direction line child GameObject of the decal. To add it, left click on the GameObject and navigate to the Inspector. Scroll down to the bottom of the Inspector, left click on “Add Component”, then search for and add the DottedLineInformationGatherer component.

Two things are required to make sure this works. One being that a LineRenderer component is required on the GameObject this component is attached to -- adding that is similar to adding any other component. The other requirement is the “End Pos" field. This field represents the end point as explained above. Drag in the other child, the end point, of the parent decal into the field.